[Passing Cars]

```
FUNCTION Solution (A)

    east_cars = 0
    Passing_pairs = 0

FOR each car in A

        IF car equals 0
            east_cars = east_cars + 1
        EISE
            Passing_pairs = passing_pairs + east_cars
        END IF

        IF passing_pairs > 1 000 000 000
            RETURN -1
        END IF

    END FOR

    RETURN passing_pairs

END FUNCTION
```

[FrogRiverOne]

```
FUNCTION Solution (X, A)

    Covered_Positions = EMPTY SET
    Covered_Count = 0

    FOR time from 0 to length(A) - 1

        Position = A[time]

        IF position is not in Covered_positions
            ADD position to Covered_positions
            Covered_Count = Covered_Count + 1

            IF Covered_Count equals X
                RETURN time
            END IF
        END IF

    END FOR

    RETURN -1


END FUNCTION
```

[Tape Equilibrium]

```
total_Sum = SUM of all elements in A
left_Sum = 0
Minimum_difference = infinity

FOR P from 1 to length(A) - 1

    left_Sum = left_Sum + A[P-1]
    Right_Sum = total_Sum - left_Sum

    difference = ABSOLUTE VALUE(left_Sum-Right_Sum)

    IF difference < Minimum_difference
        Minimum_difference = difference

    END IF

END FOR

RETURN Minimum_difference

END FUNCTION
```

2

[Frog jmp]

```
FUNCTION Solution (X, Y, D)

    IF X >= Y
        RETURN 0
    END IF

    distance = Y - X
    jumps = distance DIV D

    IF distance MOD D is not equal to 0
        jumps = jumps + 1
    END IF

    RETURN jumps

END FUNCTION
```

[CyclicRotation]

```
FUNCTION Solution (A, K)

    IF length of A equals 0
        RETURN A
    END IF


K = K MOD length of A

    REPEAT K times
        last = last element of A
        REMOVE last element from A
        INSERT last at the beginning of A
    END REPEAT


    RETURN A


END FUNCTION
```